

22.08 Algoritmos y Estructuras de Datos

Level 7: EDAoogle

Introducción

En esta práctica vamos a implementar nuestro propio buscador de páginas web.

HTML & CSS

La base de las páginas web es el lenguaje HTML (HyperText Markup Language). Un ejemplo de una página web simple:

```
<html>
  <head>
    <title>My first web page</title>
  </head>
  <body>
    <h1>Hello world</h1>
    <p>This is HTML!</p>
  </body>
</html>
```

Los documentos HTML son archivos de texto, estructurados en forma de árbol. Sus nodos se inician con un *tag de apertura*, por ejemplo **<html>**, y se terminan con un *tag de cierre*, por ejemplo **</html>**. Entre tags puede haber texto y otros nodos.

El nodo raíz de un documento HTML es siempre **html**. Suele contener dos nodos hijos: **head** y **body**.

head contiene meta-información que no se muestra directamente al usuario; por ejemplo, el título del documento (**title**).

body define el contenido del documento. El contenido puede contener nodos de cabecera (por ejemplo, **h1**), párrafo (**p**), cortes de línea (**br**), enlaces a otras páginas (**a**) o formularios que el usuario puede completar (**form** e **input**).

Los tags pueden incluir *atributos*, que permiten especificar información adicional. Un ejemplo del uso de atributos en un formulario simple:

```
<form action="/search" method="get">
  <input type="text" name="q" autofocus></input>
</form>
```

HTML no define la apariencia del documento, sólo su contenido. Para definir la apariencia podemos referir desde el archivo HTML un archivo CSS (Cascading Style Sheet).

Un ejemplo de un archivo CSS:

```
.important {  
    font-size: 2rem;  
    font-weight: bold;  
    text-align: center;  
}
```

Este ejemplo define la clase CSS **important**, que permite formatear nodos de HTML:

```
<p class="important">This is important</p>
```

HTTP

Para que un usuario pueda navegar un sitio web, debe conectarse primero a un servidor HTTP (HyperText Transfer Protocol).

Una conexión HTTP consiste en una conexión TCP/IP, en la que el cliente envía comandos que el servidor responde. Un ejemplo de una sesión HTTP:

```
GET /index.html HTTP/1.1  
Host: www.google.com  
Referer: www.google.com  
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/101.0.4951.67  
Connection: keep-alive  
[Línea en blanco]
```

El servidor responde a esta petición con la respuesta HTTP:

```
HTTP/1.1 200 OK  
Date: Fri, 12 Feb 2022 16:21:12 GMT  
Content-Type: text/html  
Content-Length: 1221
```

```
<html>  
<head>  
<title>Google</title>  
[...]  
</head>  
<body>  
[...]  
</body>  
</html>
```

Hoy en día nadie usa HTTP a secas; es inseguro porque envía los datos en texto plano. Suele usarse HTTPS, que encripta HTTP sobre el protocolo TLS.

A trabajar

Instala primero la biblioteca **libmicrohttpd** con el comando:

```
vcpkg install libmicrohttpd:[x64-windows|x64-osx|x64-linux]
```

Parte del starter code que te damos, que implementa un pequeño servidor web. El servidor responde a peticiones HTTP en la URL <http://localhost:8000> y sirve archivos estáticos desde una carpeta **www**, cuyo contenido te proveemos junto a este enunciado (las 1000 páginas más visitadas de Wikipedia).

Debes completar la clase **EDAoogleHttpRequestHandler**, que hereda de **ServeHttpRequestHandler** (un handler de peticiones HTTP que sirve archivos), que a su vez hereda de **HttpRequestHandler** (una interfaz de **HttpServer** que responde a peticiones HTTP).

Para realizar la búsqueda debes preparar un índice de búsqueda. Puedes usar **std::filesystem** (de C++17) para procesar todos los archivos que se encuentran en la carpeta **www/wiki**. Para cada documento HTML, elimina los tags y quédate sólo con el texto. Luego añade palabra a palabra al índice de búsqueda. Recomendamos que hagas esto desde el constructor de **EDAoogleHttpRequestHandler**.

Luego modifica el código de **EDAoogleHttpRequestHandler** que responde a peticiones **GET /search?q=[búsqueda]**.

El buscador debe encontrar todos los documentos cuyas palabras coincidan con las palabras que se especifican en la búsqueda. Por ejemplo, la búsqueda "**dominios anidados**" debe devolver **/wiki/Evolucion_biologica.html**.

Puedes usar **std::chrono** para medir el tiempo que la búsqueda lleva.

Bonus points

- **Optimiza.** Al fin y al cabo no quieres que las personas tengan que esperar mucho (porque sino se pasarán a Google).
- **Guarda el índice de búsqueda en disco.** Así no debe regenerarse cada vez que arrancas el servidor.